

Northern University of Business and Technology Khulna
Department of Computer Science and Engineering

[System Design & Software Architecture]

Project Title: AI-Powered Resume Analyzer and Career Copilot

Course Title: Mobile Computing Lab

Course Code: CSE4204

Submitted to:

Md. Riaz Mahmud

Assistant Professor

Department of Computer Science and Engineering
Northern University of Business and Technology, Khulna

TEAM INFORMATION SHEET

Field	Details
Official Team Name	CSE4204-8A-T05
Section	8A
Project Title	AI-Powered Resume Analyzer and Career Copilot
Team Leader Name & ID	Mahammad Hasan (ID:11220320828)
Full Member List with IDs	• Shanawaz Sakib (ID:11220320916) • Ashikur Rahman Himel (ID:11220320833) • Rasel Ratul (ID:11220320827)
GitHub Repository Link	https://github.com/mahamud894/AI_Resume_Analyzer

Project Information

The **AI-Powered Resume Analyzer and Career Copilot** is an advanced, serverless Single Page Application (SPA) designed to revolutionize how job seekers optimize their career assets. Built using **React.js (Vite)**, **TypeScript**, and **Tailwind CSS** for a highly responsive frontend, the application entirely bypasses traditional backend infrastructures (such as Node.js/Express.js) by natively embedding the **Puter.js SDK**.

As demonstrated in the core workflow architecture, all crucial operations—including user session management, secure document hosting, and high-performance Large Language Model (LLM) queries (GPT-4o/Claude 3.5 Sonnet)—are executed directly from the client side through Puter's secure cloud gateway, eliminating server maintenance costs and API key exposure risks.

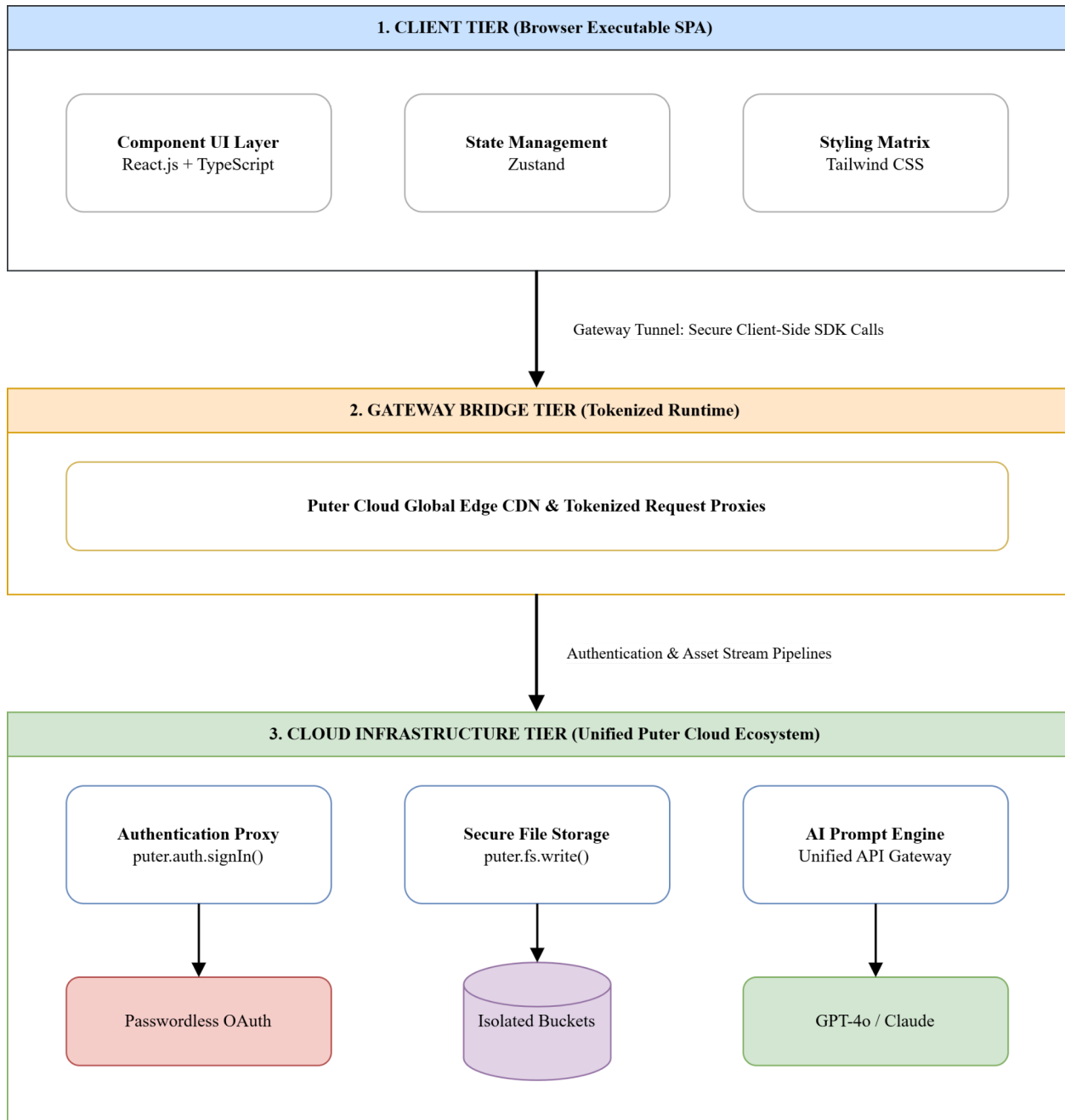
1. System Architecture Diagram

1.1 Architectural Concept & System Tiers

Unlike traditional full-stack frameworks utilizing a decoupled Node.js/Express application server and self-hosted databases, this project is engineered using a decentralized, cloud-native **Serverless Architecture** powered by the **Puter.js SDK**. The architecture consists of three core structural tiers:

- **1. Client Tier (Frontend Application Space):** Built completely as a browser-executable Single Page Application (SPA) using **React.js (Vite Core)** and **TypeScript**. Component layout hierarchies are styled using utility-first classes via **Tailwind CSS**, while all synchronous and asynchronous application states (User Sessions, Active File Caches, Evaluation Arrays) are governed by a decoupled **Zustand global store**.
- **2. Gateway Bridge Tier:** Functions as the operational pipeline through which client-side JavaScript calls are securely wrapped, validated, and proxied. It leverages the **Puter Cloud Global Edge CDN** to bypass standard local configurations and eliminate client-side API key vulnerability or exposure.
- **3. Cloud Infrastructure Tier (Unified Puter Cloud Ecosystem):** Houses the managed background servers that process application parameters. It contains dedicated sub-modules for authentication pipelines, isolated file vault binary blobs (puter.fs), and a native Large Language Model gateway interface (puter.ai) proxying advanced services like GPT-4o or Claude 3.5 Sonnet.

Fig 1: System Architecture Diagram



2. ER Diagram (Database Design)

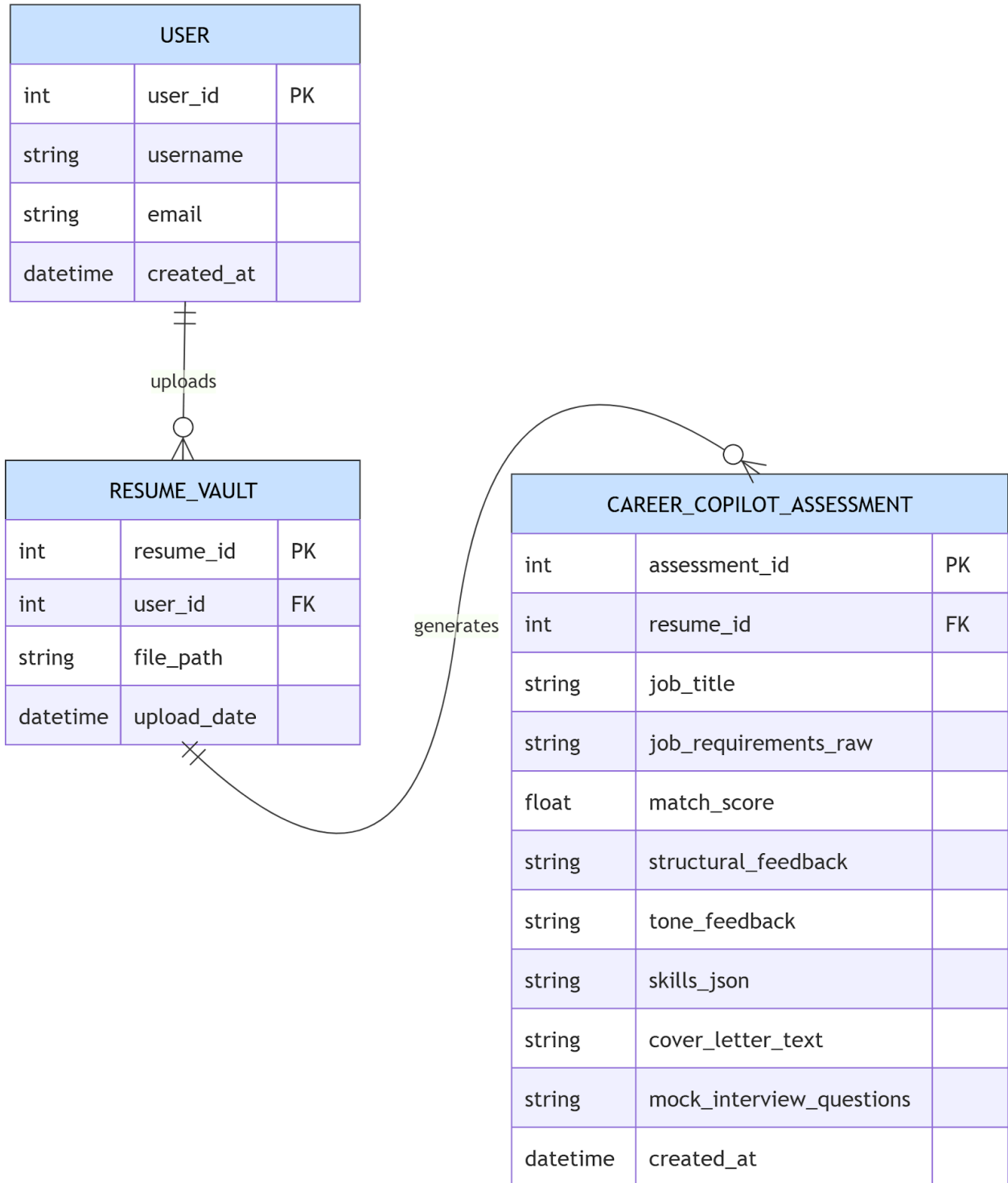
2.1 NoSQL Document & Key-Value Schema Framework

Because the application stores transactional data within **Puter NoSQL & Key-Value Storage**, it rejects strict relational schemas, table locks, and row indexing constraints. Instead, data entities are compiled into cohesive **JSON Documents** where nested objects are embedded directly into records to eliminate query latency and database joins.

The system uses three primary collections styled below in a relational equivalent structure to

satisfy academic validation rules:

NoSQL ER Diagram



2.2 Relational Mapping & Cardinality Rules

- **USER to RESUME_VAULT (\$1:N\$ Relationship):** A single registered user account mapped by a unique user_id can upload multiple historic versions of their professional credentials (resume_id) to the storage drive. Conversely, each uploaded file can belong exclusively to one account owner.
- **RESUME_VAULT to CAREER_COPILOT_ASSESSMENT (\$1:N\$ Relationship):** A static resume template entry can be run against multiple unique target job postings over time. Each execution creates a separate assessment report (assessment_id) that stores specific matching metrics, cover letters, and mock interview sheets.

3.1. Updated Use Case Diagram

Based on requirements finalized in Week 03, the system boundaries have been optimized to distinguish between user actions and background automated infrastructure tasks.

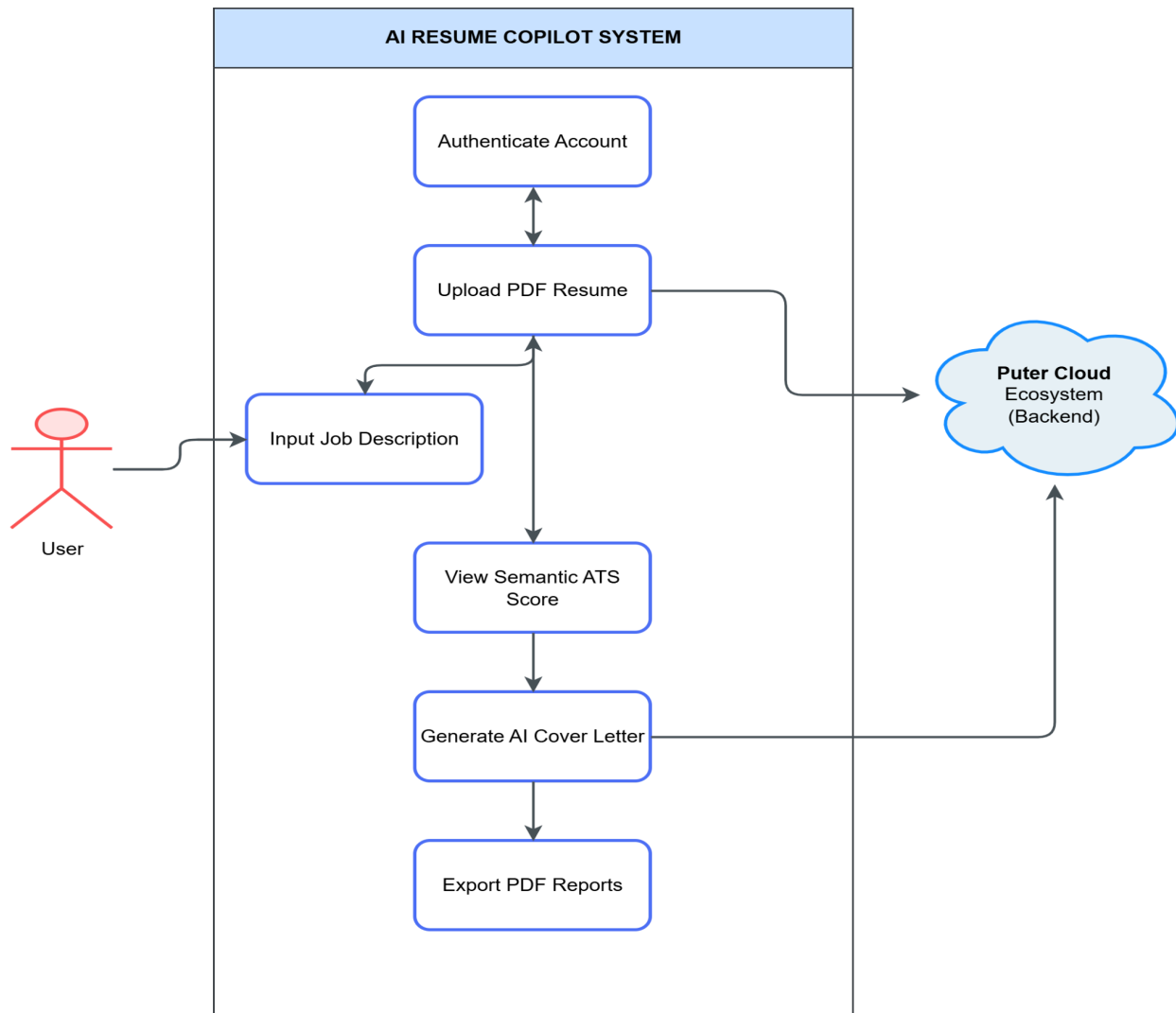


Fig 3: Use Case Diagram

- **Primary Actor:** User / Candidate (Triggers interface tasks).
- **Secondary Actor:** Puter Cloud Ecosystem (Backend Components) (Processes remote storage and handles AI queries).

3.2. Use Cases

Use Case 1: Client-Side User Login

- **Actor:** Job Seeker
- **Action:** The user clicks the "Login with Puter" action button on the landing index page.
- **System Response:** The application invokes the Puter SDK framework, triggering a secure modal popup window. Upon the user granting data permissions, the system captures a signed session token, updates the global state to **isLoggedIn: true**, and grants access to the Personal Analytics Dashboard.

Use Case 2: Resume Upload to Puter FS

- **Actor:** Job Seeker
- **Action:** The user selects or drops a raw PDF file onto the designated drag-and-drop dashboard panel.
- **System Response:** The React engine intercepts the file buffer stream, fires a **puter.fs.write('/resumes/' + unique_id + '.pdf', fileData)** routine to push the document directly to cloud storage, appends the metadata parameters into the NoSQL repository, and alerts the user with a localized success notification.

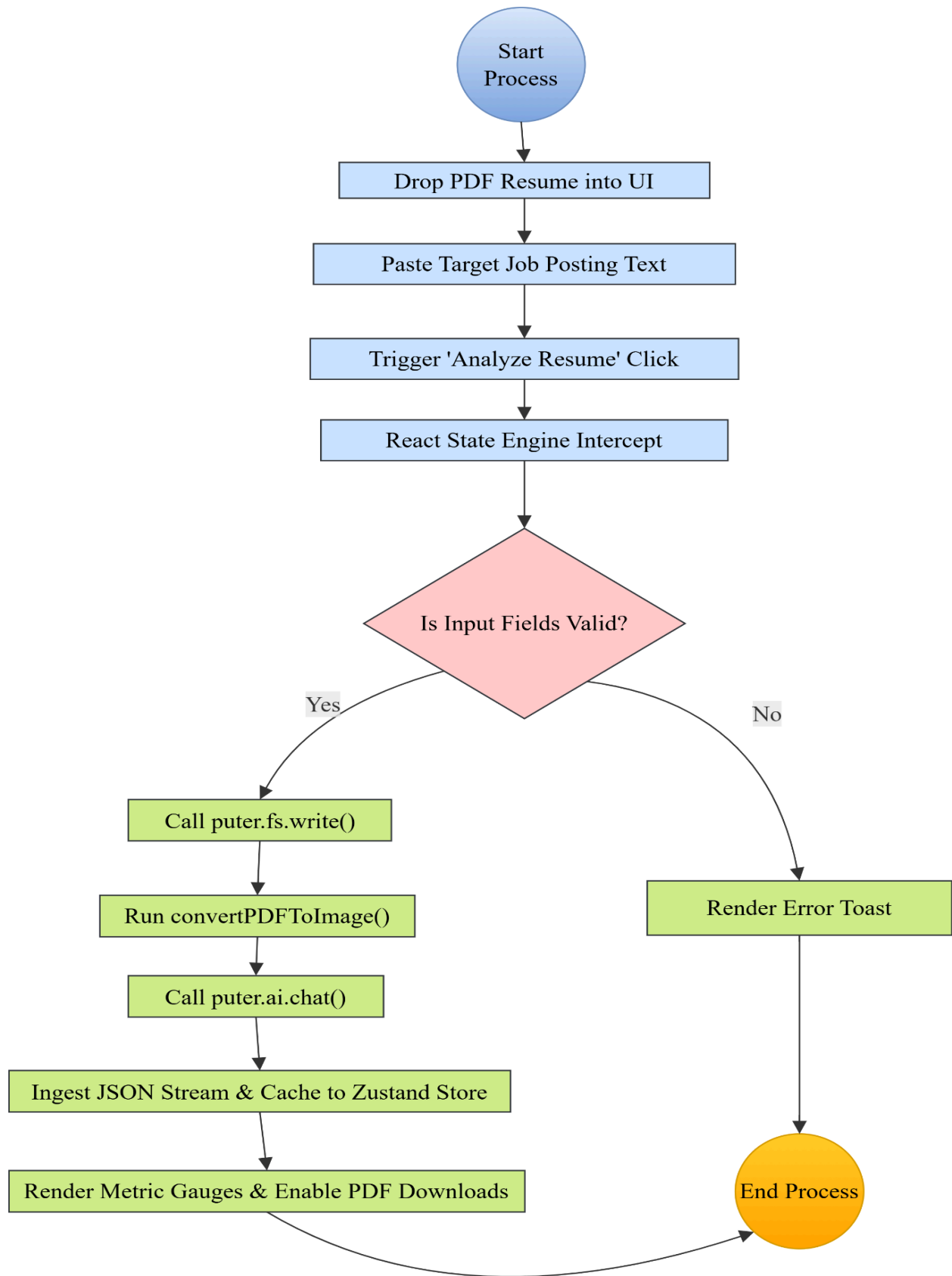
Use Case 3: Prompt Engineering & AI Audit Execution

- **Actor:** Job Seeker
- **Action:** The user highlights an active resume from their vault, enters a target job description, and triggers the "Run Diagnostic" switch.
- **System Response:** The system runs a data-mapping routine that reads the file text string alongside the job post text string, structures a system-wide optimization prompt, and dispatches it via a single asynchronous **puter.ai.chat()** function call. Once the JSON payload returns, the front-end parses it to dynamically update numerical graphs and actionable text sections on the screen.

4. Activity Diagram

The following activity diagram charts the precise workflow of the platform's core capability: **The AI Resume Audit & Semantic Analysis Workflow**.

Fig 4: User Activity Diagram



5. API Design Document

Because this application operates on a **Serverless Paradigm**, traditional HTTP network endpoint patterns (`[https://api.domain.com/v1/chat](https://api.domain.com/v1/chat)`) do not exist. All network transactions are abstracted behind method declarations inside the client-side execution framework.

5.1 `puter.auth.signIn()`

- **Purpose:** Launches a browser window to manage secure user single sign-on authentication.
- **Input Parameters:** None (Managed by Puter's authentication component dashboard UI).
- **Expected Output Object:** `Promise<UserSessionProfile>` returning a user's cloud ID (uid), registration email, and verified authentication token.

5.2 `puter.fs.write(string filePath, Blob binaryData)`

- **Purpose:** Safely writes binary document streams directly to the application's isolated cloud storage sector.
- **Input Parameters:**
 - `filePath`: Custom file path allocation string (e.g., `"/resumes/hash_id.pdf"`).
 - `binaryData`: Uncompiled application file blob captured directly from the drop interface.
- **Expected Output Object:** `Promise<FileAttributes>` containing unique item descriptors, remote web addresses, and accurate block sizes.

5.3 `puter.ai.chat(string engineeredPrompt)`

- **Purpose:** Securely connects to advanced model endpoints to process systemic textual configurations.
- **Input Parameters:** `engineeredPrompt` consisting of parsed document texts, job advertisements, and strict JSON structural formatting conditions.
- **Expected Output Object:** `Promise<StringifiedJSONPayload>` parsing matching metrics, keyword deficiencies, a tailored cover letter string, and structured prep questions.

6. AI Integration Workflow

6.1 Rationale for Artificial Intelligence Integration

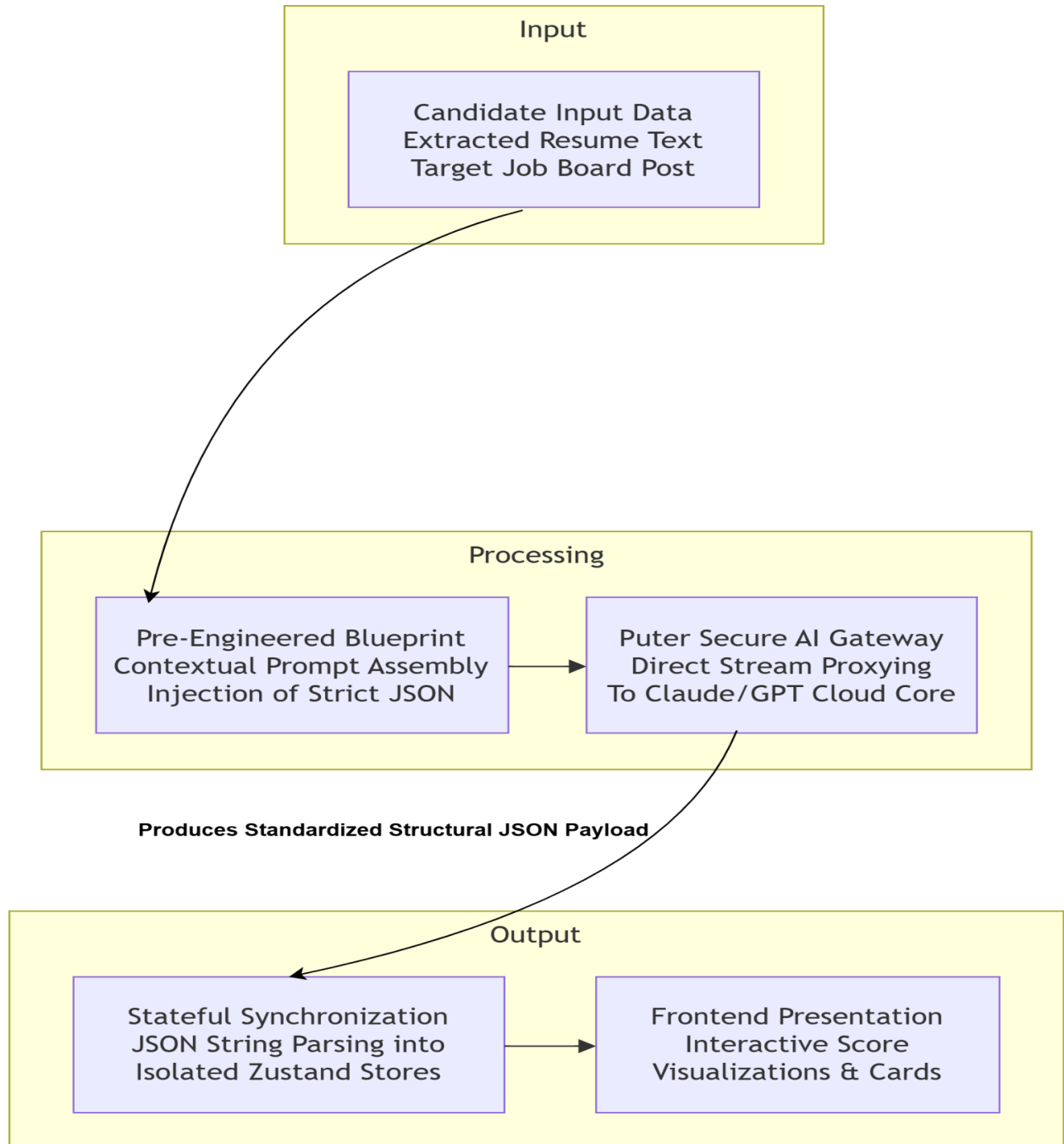
Standard keyword-checking algorithms analyze documents by looking for exact string combinations (such as counting occurrences of a word like "React"). They fail to understand conceptual skill sets, linguistic tones, structural clarity, or formatting gaps. Artificial Intelligence is mandatory to perform structural parsing, measure skill relevance against a job post, and create supporting texts (like cover letters) on demand.

6.2 Model Selections & Deployment Architecture

- **Managed System Gateway:** Puter AI Core Routing Proxy Layer.

- **Ecosystem Engine Models: Claude 3.5 Sonnet and GPT-4o.**

6.3 Technical Execution Flow Chart



6.4 Strategic Project Enhancements

Integrating AI turns a simple file repository into an intelligent career platform. Candidates receive actionable feedback within seconds, allowing them to instantly target missing technical

competencies, match resume keywords to job listings, and generate custom career assets.

7. System Constraints & Assumptions

Defining logical constraints in a design document significantly enhances the report's credibility.

- **Storage Limit:** According to the memory limitations of the Puter file system's free-tier bucket, each user can upload a PDF resume of up to **20MB**.
- **Network Runtime Dependency:** As the application is a fully serverless Single Page Application (SPA), the speed of AI report processing or data rendering depends entirely on the Puter Cloud Edge network and the response time of OpenAI/Anthropic servers.
- **Input Validation:** The quality and accuracy of the generated report will depend on the depth and quality of the raw text provided by the user in the job description.

8. References

1. Puter.js SDK Client-Side Operating Framework Manuals (<https://docs.puter.com>).
2. React Router V7 Application Routing and Lifecycle Rules (<https://reactrouter.com>).
3. Zustand Minimalist Stateful Flux Management Guidelines (<https://zustand-demo.pmnd.rs>).
4. Project Configuration Blueprint: CSE4204_8A_T05_Week02.docx.pdf.
5. Application Functional Mandates: CSE4304_8A_T05_SRS.pdf.